

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 2 Chapter No. 1 Lecture No. 11

Topic : Concept of redundancy, failover mechanisms, active-active vs.
active-passive configurations, designing for failure.

Sanjay Tiwari, E18823 Senior Technical Trainer

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

Virtualization can be implemented using multiple approaches.

Full virtualization allows unmodified operating systems to run.

Paravirtualization enhances performance through OS-level
cooperation.

Hardware-assisted virtualization improves execution efficiency.

Hypervisors control and manage virtual machines.

Type 1 hypervisors operate directly on physical hardware.

Type 2 hypervisors run on top of a host operating system.

Virtualization architecture determines isolation and control mechanisms.

Different use cases require different hypervisor choices.

Selecting the right virtualization type improves overall system efficiency.

RECAP

Learning Objectives

Availability and reliability overview

Concept of redundancy

Types of redundancy

Failover mechanisms

Failure detection techniques

Active-active configurations

Active-passive configurations

Designing systems for failure

1

Availability ensures continuous service access

Reliability ensures consistent correct operation

Failures are inevitable in large systems

Hardware, software, and network can fail

Design must assume component failures

Goal is service continuity, not zero failure High availability minimizes downtime impact Core principle in modern system design

Availability and Reliability Overview

Redundancy means duplication of components

Prevents single point of failure

Ensures backup resources are available

Can be applied at hardware level

Can be applied at software level

Can be applied at network level

Enables seamless fault tolerance

Increases system resilience

Concept of Redundancy

Source: [geeksforgeeks](#)

Redundancy

Hardware redundancy (servers, disks)

Network redundancy (links, switches)

Power redundancy (UPS, generators)

Data redundancy (replication, mirroring)

Application redundancy (multiple instances)

Geographic redundancy (multiple locations)

Load redundancy across resources

Control-plane redundancy

Types of Redundancy

Eliminates single points of failure in systems

Ensures continuous service availability during component failure

Allows planned maintenance without service interruption

Improves fault tolerance across infrastructure layers

Enhances reliability of mission-critical applications

Supports load distribution under peak demand

Increases overall system resilience

Enables rapid recovery from unexpected faults

Benefits of Redundancy

Failover is the automatic switching to backup resources

Triggered when primary component becomes unavailable

Works closely with redundancy mechanisms

Ensures uninterrupted service delivery

Can be automated or operator-controlled

Relies on real-time health monitoring

Minimizes impact of failures on users

Essential for high-availability architectures

Failover Mechanism Overview

Continuous heartbeat signals between system nodes

Periodic health-check probes for responsiveness
Timeout-based detection of unresponsive services

They Track the following Parameters Periodically:

Monitoring CPU, memory, and I/O anomalies

Application-level health indicators

Network reachability and latency checks

Load balancer-based health evaluation

Alerting systems for abnormal behavior

Failure Detection Techniques

Source: Algomaster

Heartbeat Method

Failure condition identified by monitoring systems

Affected component marked as unhealthy

Traffic or workload redirected automatically

Standby or alternate component activated

Session or state continuity ensured if applicable
System load redistributed evenly

Monitoring validates post-failover stability

Normal operations restored with minimal disruption

Failover Execution Process

Multiple components serve requests simultaneously

Traffic distributed across all active nodes

No idle standby resources present

Failure of one node reduces capacity, not service

Requires coordination and synchronization

Designed for high throughput environments

Common in large-scale distributed systems

Maximizes infrastructure utilization

Active-Active Configuration Concept

Source: [geeksforgeeks](#)

Active Active

Near-zero downtime during failures

Efficient utilization of all available resources

Improved response time and throughput

Seamless handling of traffic spikes

Automatic fault tolerance without manual intervention

Suitable for global and distributed workloads

Supports horizontal scalability

Enhances overall system performance

Advantages of Active-Active

One component actively handles workload

Secondary component remains on standby

Passive node activates only during failure

Simplifies synchronization requirements

Easier to design and maintain

Suitable for predictable workloads

Often used in smaller deployments

Provides controlled failover behavior

Active-Passive Configuration Concept

Source: [geeksforgeeks](https://www.geeksforgeeks.org/active-passive-configuration/)

Active Passive

Simpler architecture and management

Easier failure diagnosis and troubleshooting

Lower synchronization complexity

Clear separation of active and backup roles

Reduced operational overhead

Predictable recovery process

Lower initial configuration effort

Suitable for less dynamic environments

Advantages of Active-Passive

Active-Active vs Active-Passive

Feature

Active-Active

Active-Passive

Core Architecture

Multiple components serve requests simultaneously.

One component actively handles the workload, while the secondary remains on standby.

Resource Utilization

Efficient utilization of all available resources (no idle standby resources present) .

Lower utilization, as the passive node activates only during failure.

Primary Priority

Prioritizes performance and availability.

Prioritizes simplicity and control.

Complexity & Maintenance

Higher operational complexity. Requires strong coordination and synchronization mechanisms.

Simpler architecture and management. Lower synchronization complexity and easier failure diagnosis.

Failover Behavior

Automatic fault tolerance with near-zero downtime. A node failure reduces capacity, not service.

Predictable and controlled failover behavior that requires clear failover triggers.

Best Suited For

High-throughput environments , large-scale distributed systems , and seamlessly handling traffic spikes.

Predictable workloads and smaller deployments needing lower initial configuration effort.

Source: [geeksforgeeks](#)

Active Active vs Active Passive

Systems must assume inevitable failures Design focuses on recovery, not prevention Components expected to fail independently Architecture must tolerate partial outages Failure should not cascade across system Services should degrade gracefully

Automated recovery is preferred

Core philosophy of resilient systems

Designing for Failure Concept

Loose coupling between system components

Stateless or minimally stateful designs

Use of retries with backoff strategies

Timeout handling to avoid blocking

Isolation of faulty components

Graceful error handling mechanisms

Continuous monitoring and alerting

Regular validation of system health

Failure-Tolerant Design Principles

Source: PULSE

Fault Tolerance Design Example

Distribute incoming requests evenly

Detect unhealthy components automatically

Remove failed nodes from traffic flow

Enable smooth failover operations

Prevent overload on individual components

Improve overall system responsiveness

Act as entry point for users

Support redundancy and availability goals

Role of Load Balancers

Increased system design complexity

Higher infrastructure and operational cost

Difficulty in ensuring data consistency

Complex debugging during failure scenarios

Need for accurate monitoring systems

Configuration errors can cause outages

Failover testing is non-trivial

Requires disciplined operational practices

Challenges in High Availability

Online banking and payment systems

E-commerce transaction platforms

Telecommunications switching systems

Healthcare patient information systems Online

learning and examination portals Enterprise

business-critical applications Media streaming

and broadcasting services Government digital

service platforms

Real-World Use Cases

Source: Narmi

Cloud-Native Banking Systems Use Case

Summary of the Lecture

Failures are unavoidable in modern systems

Redundancy forms the foundation of availability

Failover mechanisms ensure continuity

Active-active maximizes performance

Active-passive simplifies reliability design

Designing for failure improves resilience

High availability protects user experience

Essential principle in modern infrastructure

6

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview

Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview

Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview

Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

10

Next Lecture

DR strategies (Backup & Restore, Pilot Light, Warm Standby, Multisite), RTO (Recovery Time Objective), RPO (Recovery Point Objective)

7

Quiz/ FAQ's

1. What is redundancy in system design?

Redundancy is the practice of duplicating critical system components so that if one component fails, another can take over, ensuring continuous system availability and avoiding single points of failure.

2. How does a failover mechanism work?

Failover mechanisms continuously monitor system health, detect failures, isolate faulty components, and automatically redirect traffic or workload to healthy backup components to maintain service continuity.

3. What is the difference between active-active and active-passive configurations?

In active-active configuration, all nodes actively handle traffic simultaneously, while in active-passive configuration, one node handles traffic and the other remains on standby until a failure occurs.

4. Why is designing for failure important in modern systems?

Designing for failure acknowledges that hardware, software, and networks will eventually fail and focuses on building systems that can tolerate, recover, and continue operating despite such failures.

5. What are common challenges in implementing high availability?

Key challenges include increased system complexity, higher operational costs, difficulty in failure testing, synchronization issues, and the need for robust monitoring and automation.

Google Site Reliability Engineering (SRE) Book

<https://sre.google/sre-book/availability-table/>

Microsoft Azure – High Availability Concepts

[https://learn.microsoft.com/en-us/azure/architecture/framework/
resiliency/overview](https://learn.microsoft.com/en-us/azure/architecture/framework/resiliency/overview)

AWS Well-Architected Framework – Reliability Pillar

[https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/
welcome.html](https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html)

IBM – High Availability and Failover Concepts

<https://www.ibm.com/docs/en/aix/7.3?topic=availability-high> Red Hat –
Designing for Failure

<https://www.redhat.com/en/topics/cloud-native-apps/designing-for-failure>

9

Class-Wise Feedback

11

Thank You

For queries

Email: sanjay.e18823@cumail.in

12